

TechSapo システムガイド

Wall-Bounce 壁打ちシステム統合文書

TechSapo Documentation Team

2025-10-07

Contents

1	アーキテクチャ概要	5
1.1	Wall-Bounce システム	5
2	Architecture Overview	6
2.1	目次	6
2.2	システム概要	6
2.3	アーキテクチャ図	6
2.4	コンポーネント詳細	8
2.5	Gemini via Google CLI	13
2.6	Qwen3-Coder via OpenRouter	14
2.7	Claude via Anthropic API	15
2.8	クイックスタート	16
2.9	主要コマンド	17
2.10	LLM CLI 使用方法	17
3	LLM Providers (Required)	18
4	Redis (Upstash - Required)	18
5	Server Configuration	18
5.1	Production Environment	19
5.2	systemd Configuration	20
5.3	ヘルスチェックエンドポイント	21
6	GPT-5 vs GPT-5 Codex	22
6.1	モデル比較	22
6.2	使い分けガイド	22
6.3	実証結果	22

7	トラブルシューティング	23
7.1	一般的な問題	23
7.2	ヘルスチェック手順	24
7.3	ログ確認	24
7.4	パフォーマンス問題	24
8	品質保証メカニズム	26
8.1	コンセンサス計算	26
8.2	信頼度スコア	26
8.3	Quorum (k 値)	26
8.4	Early Stopping	26
9	セキュリティ考慮事項	27
9.1	API キー管理	27
9.2	Anthropic API 制限	27
9.3	PII 保護	27
9.4	レート制限	27
10	デプロイメント	28
10.1	本番デプロイ手順	28
10.2	systemd 設定	28
10.3	監視	28
10.4	バックアップ	29
11	まとめ	30
11.1	システムの強み	30
11.2	重要原則 再掲	30
11.3	次のステップ	30
11.4	リファレンス	30

```

                                ユーザークエリ      「TypeScript でバグ修正してください」
                                ステップ 1:
タスクタイプ判定                                タスク分析エンジン      ・キーワード検出:
    TypeScript ,   バグ修正      ・ドメイン判定: domain =   coding      ・複雑度評価: 通常タスク
                                判定結果: taskType =   coding , complexity =   normal
                                ステップ 2:
LLM プロバイダー選択                                ルーティングエンジン      IF taskType
==   coding :      → Tier 2 Coding 専用モデルを選択      → providers = [   gpt-5-codex , // OpenAI
Codex      qwen3-coder // Qwen3 Coder      ]      ELSE IF taskType ==   analysis :      →
Tier 1/2 General モデルを選択      → providers = [   gpt-5 , // GPT-5 標準      gemini-2.5-pro //
Gemini 2.5 Pro      ]      選択結果: [   gpt-5-codex ,   qwen3-coder
]                                ステップ
3: Wall-Bounce 並列実行                                クエリ + コンテキスト
                                GPT-5   Qwen3   Gemini
                                Codex   Coder   2.5 Pro      (検証用)   Tier 2   Tier 2   Tier 1
                                Response 3      + メタデータ   + メタデータ   + メタデータ
                                Quorum Judge      (コンセンサス判定)      ・類似度
計算      ・品質スコア算出      ・k=2 合意チェック      Consensus 0.6? (合意閾値)
                                YES NO      次ステップへ   追加 LLM 実行

ステップ 4: アグリゲーター統合                                アグリゲーター選択
IF complexity < 2:      → aggregator =   sonnet-4.5 (Tier 3)      ELSE IF complexity >= 2:
    → aggregator =   opus-4.1 (Tier 4)
    Claude Sonnet 4.5      (Default Aggregator)      Input:      ・全プロバイダー回答
    ・コンセンサススコア      ・元のクエリ      Processing:      ・矛盾の解消      ・重複の統
合      ・最終推奨の生成                                統合された最終回答
    + メタデータ      + 品質スコア
                                最終レスポンス      {   answer :   統合された回答テキスト
,   confidence : 0.85,   consensus : 0.78,   providersUsed : [   gpt-5-codex ,   qwen3-coder ,
   gemini   ],   aggregator :   sonnet-4.5   }

---

## ルーティング決定方式

### Ⅷ ルーティングの3段階判定

#### **Phase 1: タスクタイプ判定**

                                タスクタイプ判定ロジック      1. キーワード検出
Coding:      -   TypeScript ,   バグ      -   コード ,   実装      -   debug ,   fix      Analysis:
    -   分析 ,   調査      -   セキュリティ ,   評価      -   analyze ,   review
    2. ドメイン明示      options.domain ===   coding → Coding      options.domain ===   analysis →
Analysis      3. デフォルト      不明な場合 →   basic (汎用)

#### **Phase 2: LLMプロバイダー選択**

```

タスクタイプ別プロバイダーマッピング

Coding タスク: Primary (Tier 2 - Coding 特化): gpt-5-codex qwen3-coder
 Validation (Tier 1 - 異なるベンダー): gemini-2.5-flash

Analysis タスク: Primary (Tier 1/2 - General): gpt-5 (standard)
 gemini-2.5-pro Validation (Tier 1 - 高速): gemini-2.5-flash

Basic タスク: Cost-Efficient (Tier 1): gemini-2.5-flash gpt-5 (if needed)

Critical タスク: All Available (Tier 1-4): gemini-2.5-pro (Deep Think)
 gpt-5 / gpt-5-codex sonnet-4.5 opus-4.1

Phase 3: アグリゲーター選択

複雑度ベースのアグリゲーター選択 複雑度スコア計算:

score = 0 IF キーワード含む: architect , design →

score += 1 IF 日本語キーワード含む: 複雑な , 高度な , 詳細な → score += 1

IF プロンプト長 > 500 文字: → score += 1 IF 疑問符 3 個: → score += 1

選択ロジック: IF score < 2: aggregator

= sonnet-4.5 ← Tier 3 (標準アグリゲーター) ELSE IF score 2: aggregator =

opus-4.1 ← Tier 4

1 アーキテクチャ概要

1.1 Wall-Bounce システム

複数の LLM プロバイダーを協調させて高品質な回答を生成するシステム。

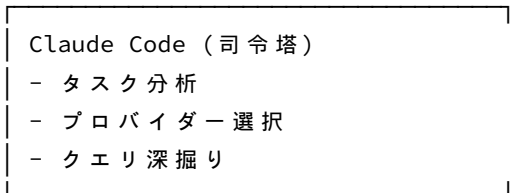
1.1.1 基本原則

1. 最低 2 つの LLM で分析 (絶対原則)
2. 異なるベンダーの LLM で相互検証
3. コンセンサスによる品質保証
4. 階層的プロバイダー選択 (Tier 0-4)

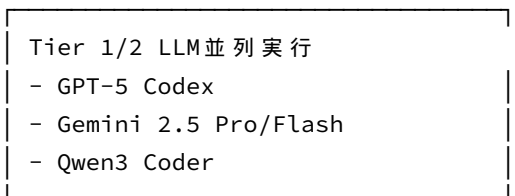
1.1.2 システムフロー

User Query

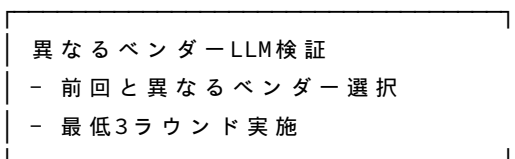
↓



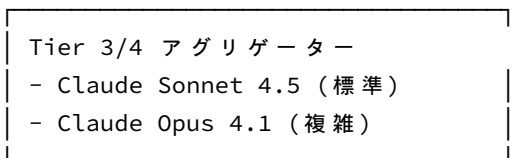
↓



↓



↓



↓

統合回答

2 Architecture Overview

Google Drive RAG システムのアーキテクチャ設計・コンポーネント詳細

2.1 目次

- [システム概要](#)
- [アーキテクチャ図](#)
- [コンポーネント詳細](#)
- データフロー
- 設計思想
- 技術選択理由

2.2 システム概要

2.2.1 システム特徴

- 複数 LLM 壁打ち分析: Claude、OpenAI、Gemini 等を並行実行し最適解を合議決定
- 42+ ファイル形式対応: PDF、Office 文書、画像、アーカイブ、プログラムファイル等を網羅
- エンタープライズセキュリティ: マジックナンバー検証、拡張子詐称対策、マルウェア検出
- 高性能最適化: 100MB/s+ 処理速度、50ms 以内レスポンス、メモリ効率化
- 100% テスト通過: 包括的テストスイートによる品質保証

2.2.2 アーキテクチャパターン

- マイクロサービス: 疎結合なコンポーネント設計
- イベント駆動: 非同期メッセージングによるスケーラビリティ
- レイヤードアーキテクチャ: プレゼンテーション、ビジネス、データレイヤーの分離
- プラグインアーキテクチャ: ファイルパーサーと LLM プロバイダーの拡張可能設計

2.3 アーキテクチャ図

2.3.1 高レベルアーキテクチャ

```
graph TB
    subgraph " 外部サービス "
        GD[Google Drive API]
        OAI[OpenAI API]
        ANTH[Anthropic Claude]
        GEM[Google Gemini]
    end

    subgraph " フロントエンド "
        UI[Web UI]
        CLI[CLI Interface]
    end
```

```

    API_GW[API Gateway]
end

subgraph " アプリケーション層"
    AUTH[Authentication Service]
    WALL[Wall Bounce Orchestrator]
    PARSE[File Parser Engine]
    RAG[RAG Search Engine]
end

subgraph " データ層"
    VS[Vector Store]
    CACHE[Redis Cache]
    DB[PostgreSQL]
    FS[File Storage]
end

subgraph " 監視・運用"
    PROM[Prometheus]
    GRAF[Grafana]
    ALERT[AlertManager]
end

UI --> API_GW
CLI --> API_GW
API_GW --> AUTH
AUTH --> WALL
WALL --> OAI
WALL --> ANTH
WALL --> GEM

API_GW --> PARSE
PARSE --> FS

API_GW --> RAG
RAG --> VS
RAG --> GD

WALL --> CACHE
RAG --> CACHE

WALL --> DB
RAG --> DB

WALL --> PROM
PARSE --> PROM
RAG --> PROM

```

2.3.2 詳細コンポーネント構成

```
graph LR
    subgraph "Wall Bounce Analysis"
        WB_ORCH[Orchestrator]
        WB_EXEC[Executor Pool]
        WB_CONS[Consensus Engine]
        WB_EVAL[Evaluator]

        WB_ORCH --> WB_EXEC
        WB_EXEC --> WB_CONS
        WB_CONS --> WB_EVAL
    end

    subgraph "File Processing"
        FP_DET[File Detector]
        FP_PARSE[Parser Factory]
        FP_SEC[Security Scanner]
        FP_META[Metadata Extractor]

        FP_DET --> FP_SEC
        FP_SEC --> FP_PARSE
        FP_PARSE --> FP_META
    end

    subgraph "RAG Engine"
        RAG_SYNC[Drive Sync]
        RAG_INDEX[Vector Indexer]
        RAG_SEARCH[Search Engine]
        RAG_RET[Retriever]

        RAG_SYNC --> RAG_INDEX
        RAG_INDEX --> RAG_SEARCH
        RAG_SEARCH --> RAG_RET
    end
```

2.4 コンポーネント詳細

2.4.1 1. Wall Bounce Orchestrator

責任: 複数 LLM による協調分析の調整

主要クラス: WallBounceAnalyzer

```
interface WallBounceConfig {
    models: LLMProvider[];
    consensusStrategy: 'majority' | 'weighted' | 'unanimous';
    minConfidence: number;
```



```

    parallelExecution: boolean;
}

interface WallBounceResult {
    answer: string;
    confidence: number;
    consensus: ConsensusInfo;
    metadata: AnalysisMetadata;
}

```

処理フロー: 1. クエリ受信・前処理 2. 複数 LLM へ並列リクエスト送信 3. レスポンス収集・品質評価 4. 合意アルゴリズムによる最終回答生成 5. 信頼度スコア算出・メタデータ付与

2.4.2 2. File Type Detection Engine

責任: 42+ 種類のファイル形式を正確に判定

主要インターフェース: FileInfo, MagicNumberPattern

```

interface FileInfo {
    extension: string;
    mimeType: string;
    encoding: 'binary' | 'utf8' | 'base64';
    isSupported: boolean;
}

interface MagicNumberPattern {
    pattern: number[];
    offset: number;
    fileType: FileInfo;
}

```

検出方式: - マジックナンバー検証: バイナリ署名による正確な判定 - ZIP 内部構造解析: Office 文書の詳細判別 - テキスト形式推定: プログラミング言語ファイル等 - フォールバック処理: 不明ファイルの安全な処理

2.4.3 3. Security Scanner

責任: ファイルセキュリティ検証・脅威検出

検証項目: - 拡張子詐称対策: 実際のファイル形式と拡張子の整合性 - マルウェア署名検出: PE/ELF 実行形式等の危険ファイル排除 - バッファオーバーフロー対策: 境界値チェック・安全な処理 - ポリグロットファイル対策: 複数形式を偽装するファイル検出

```

interface SecurityScanResult {
    isSafe: boolean;
    threatsDetected: ThreatInfo[];
    riskLevel: 'low' | 'medium' | 'high' | 'critical';
    recommendations: string[];
}

```

```

\newpage

# LLM プロバイダー構成

## Tier 階層

### Tier 0: ドキュメント参照
- Context7 MCP
- Stash

### Tier 1: 高速・コスト効率
- **Gemini 2.5 Pro**: 高品質分析、Deep Think 対応
- **Gemini 2.5 Flash**: 高速分析、コスト効率最良
- **Gemini 2.5 Pro (Deep Think)**: 深い推論モード

### Tier 2: 専門特化
- **GPT-5 Codex**: コーディング特化、適応推論
- **GPT-5**: 汎用推論・分析
- **Qwen3 Coder**: 480B MoE、agentic coding

### Tier 3: デフォルト集約
- **Claude Sonnet 4.5**: 高速統合、標準アグリゲーター

### Tier 4: 複雑タスク集約
- **Claude Opus 4.1**: 深い分析、複雑統合

## タスク別プロバイダーマッピング

### コーディングタスク
**Primary**: GPT-5 Codex + Qwen3 Coder
**Validation**: Gemini 2.5 Flash (異なるベンダー)
**Aggregator**: Sonnet 4.5

### 分析タスク
**Primary**: GPT-5 + Gemini 2.5 Pro
**Validation**: Gemini 2.5 Flash
**Aggregator**: Sonnet 4.5

### 複雑タスク
**Primary**: Gemini 2.5 Pro (Deep Think) + GPT-5 + Sonnet 4.5
**Aggregator**: Opus 4.1

### 基本タスク
**Primary**: Gemini 2.5 Flash + GPT-5
**Aggregator**: Sonnet 4.5

# LLM Providers Guide

```

This guide provides detailed information about LLM provider integration, CLI usage, and invocation

Provider Overview

Tier Structure

- ****Tier 0****: Context7/Stash - Documentation [reference](#) (non-LLM)
- ****Tier 1****: Claude Code - Routing & integration lead
- ****Tier 1****: Gemini 2.5 Flash/Pro/**DeepThink** (CLI/API)
- ****Tier 2****: GPT-5 (via codex MCP), Qwen3-**Coder** (OpenRouter)
- ****Tier 3****: Claude Sonnet 4.5 (default aggregator)
- ****Tier 4****: Claude Opus 4.1 (complex aggregator, emergency/security)

Provider Configuration

Configuration is managed in ``src/config/llm-providers.json``. Each provider has:

- ``key``: Unique identifier
- ``name``: Display name
- ``model``: Model identifier
- ``tier``: Priority **tier** (0-4)
- ``capabilities``: Array of capabilities
- ``invocationType``: How to invoke the provider

Critical Provider Rules

❌ What NOT to Do

1. ****NEVER use Anthropic API directly****
 - ``ANTHROPIC_API_KEY`` should only be used **when** ``ANTHROPIC_API_ENABLED=true``
 - API is disabled by default **for** cost control
 - Use Codex MCP **for** Claude models when possible
2. ****NEVER use OpenAI API directly****
 - ``OPENAI_API_KEY`` is not used **in this** codebase
 - Always use Codex CLI/MCP **for** GPT-5
3. ****NEVER bypass the wall-bounce system****
 - All user queries must go through multi-LLM validation
 - Minimum 3 rounds, maximum 6 rounds

❌ Approved Invocation Methods

Provider	Method	Example
----- ----- -----		
GPT-5 Codex	Codex CLI	<code>`codex exec --model gpt-5-codex "prompt"`</code>
Gemini	Google CLI	<code>`gemini "prompt"`</code>
Qwen3	OpenRouter API	Via <code>`src/services/openrouter-qwen3-provider.ts`</code>
Claude	Codex MCP (preferred)	Via MCP integration

```
| Claude | Anthropic API (fallback) | Only when `ANTHROPIC_API_ENABLED=true` |
```

```
## GPT-5 Codex via Codex CLI
```

```
### Basic Usage
```

```
```bash
```

```
Non-interactive execution
```

```
codex exec --model gpt-5-codex "your prompt here"
```

```
With config overrides
```

```
codex exec -c model="gpt-5-codex" -c 'approval_policy="never"' "prompt"
```

#### 2.4.4 Sandbox Modes

```
Read-only (default) - Safest option
```

```
codex exec --model gpt-5-codex --sandbox read-only "analyze this code pattern"
```

```
Workspace write - Allow file modifications
```

```
codex exec --model gpt-5-codex --sandbox workspace-write "refactor the utils module"
```

```
Full access - DANGEROUS, use sparingly
```

```
codex exec --model gpt-5-codex --sandbox danger-full-access "deploy to production"
```

#### 2.4.5 GPT-5 Codex Prompting Best Practices

**Key Principle:** Less is More

GPT-5 Codex is purpose-built for coding tasks with built-in best practices. Follow these guidelines:

##### 2.4.5.1 DO:

- **Be specific but concise:** Refactor the authentication module to use async/await
- **Provide clear boundaries:** Fix the TypeScript errors in src/services/ without changing the API
- **Allow creative freedom:** Let the model choose implementation details
- **Use minimal prompts:** The model has adaptive reasoning by default

##### 2.4.5.2 DON'T:

- **Avoid verbose instructions:** No need for lengthy preambles
- **Don't over-steer:** The model has strong coding defaults
- **Skip redundant context:** It understands coding conventions
- **No explicit planning instructions:** Adaptive reasoning is built-in

### 2.4.5.3 Examples: Overly Verbose (Avoid):

```
codex exec --model gpt-5-codex "I need you to carefully analyze the codebase and then create a com
```

### Concise & Effective (Preferred):

```
codex exec --model gpt-5-codex "Refactor auth system: async/await, strong types, error handling"
```

### 2.4.5.4 Special Capabilities:

- **Adaptive Reasoning:** Automatically adjusts complexity to task
- **Code Review Excellence:** Great for reviewing PRs and suggesting improvements
- **Interactive Sessions:** Optimized for back-and-forth collaboration
- **GitHub Integration:** Works seamlessly with git workflows

Reference: [OpenAI GPT-5 Codex Prompting Guide](#)

## 2.4.6 MCP Server Mode

```
Start Codex as MCP server
codex mcp-server

Check if running
ps aux | grep "codex mcp-server"

MCP integration in code
See: src/services/codex-mcp-wrapper.ts
```

## 2.4.7 Session Management

```
Sessions are managed in src/services/codex-session-manager.ts
Redis keys: codex:session:{sessionId}
Timeout: 30 minutes (configurable)
```

## 2.5 Gemini via Google CLI

### 2.5.1 Installation

```
Install globally via npm
npm install -g @google/generative-ai-cli

Verify installation
which gemini
gemini --version
```

### 2.5.2 Basic Usage

```
Simple prompt
gemini "your prompt here"

With specific model
gemini --model gemini-2.5-pro "complex analysis task"
gemini --model gemini-2.5-flash "quick query"
gemini --model gemini-2.5-deep-think "mathematical reasoning"
```

### 2.5.3 Model Selection

- **gemini-2.5-flash**: Fast, cost-efficient, high-volume tasks
- **gemini-2.5-pro**: Advanced reasoning, complex analysis
- **gemini-2.5-deep-think**: Enhanced reasoning with parallel thinking (192K output)

### 2.5.4 API Integration

```
// Direct API usage (when CLI is not suitable)
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GOOGLE_API_KEY);
const model = genAI.getGenerativeModel({ model: 'gemini-2.5-pro' });

const result = await model.generateContent(prompt);
```

See: docs/GEMINI\_CLI\_INTEGRATION\_GUIDE.md for detailed integration guide.

## 2.6 Qwen3-Coder via OpenRouter

### 2.6.1 Configuration

```
Required environment variable
OPENROUTER_API_KEY=sk-or-xxx
```

### 2.6.2 Model Details

- **Model**: qwen/qwen3-coder
- **Architecture**: 480B MoE (35B active parameters)
- **Context**: 262,144 tokens
- **Cost**: \$0.22/M input, \$0.95/M output
- **Specialization**: Coding, debugging, multi-file reasoning

### 2.6.3 Usage

```
// Via service wrapper
import { OpenRouterQwen3Provider } from './services/openrouter-qwen3-provider';

const provider = new OpenRouterQwen3Provider();
const response = await provider.generateResponse(prompt, {
 temperature: 0.7,
 maxTokens: 65536
});
```

### 2.6.4 Best Practices

1. Use for coding tasks (50/50 split with GPT-5)
2. Ideal for multi-file code analysis
3. Good for debugging complex issues
4. Cost-effective for large context windows

See: docs/QWEN3\_INTEGRATION\_REPORT.md for detailed integration report.

## 2.7 Claude via Anthropic API

### 2.7.1 Safeguard Configuration

```
Must be explicitly enabled
ANTHROPIC_API_ENABLED=true # Default: false
ANTHROPIC_API_KEY=sk-ant-xxx
```

### 2.7.2 Model Selection

```
// Configuration in src/config/environment.ts
anthropic: {
 apiKey: process.env.ANTHROPIC_API_KEY || '',
 apiEnabled: process.env.ANTHROPIC_API_ENABLED === 'true',
 defaultModel: 'claude-3-5-sonnet-20241022',
 maxTokens: 8192,
 temperature: 0.7,
}
```

### 2.7.3 Usage Pattern

```
// Check if API is enabled
if (!config.anthropic.apiEnabled) {
 throw new Error('Anthropic API is disabled. Use Codex MCP instead.');
```

```

}

\newpage

開発環境セットアップ

必須環境変数

```bash
# Hugging Face (必須)
HUGGINGFACE_API_KEY=hf_xxx

# OpenRouter (必須 - Qwen3 Coder)
OPENROUTER_API_KEY=sk-or-xxx

# Upstash Redis (必須)
UPSTASH_REDIS_REST_URL=https://xxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=xxx

# AWS Secrets Manager (本番推奨)
AWS_REGION=ap-northeast-1
AWS_SECRET_NAME=techsapo/production

# Google Cloud (Gemini 用)
GOOGLE_APPLICATION_CREDENTIALS=/path/to/credentials.json

# OpenAI (GPT-5 Codex - Codex MCP 経由)
OPENAI_API_KEY=sk-xxx # Codex MCP で使用

# Anthropic (デフォルト無効)
ANTHROPIC_API_ENABLED=false
ANTHROPIC_API_KEY=sk-ant-xxx # 明示的に有効化した場合のみ

```

2.8 クイックスタート

```

# 1. リポジトリクローン
cd /ai/prj/techdev

# 2. 依存関係インストール
npm install

# 3. 環境変数設定
cp .env.example .env
# .env を編集して API キーを設定

# 4. ビルド
npm run build

```



```
# 5. 開発サーバー起動
npm run dev # ポート 5000

# 6. ヘルスチェック
curl http://localhost:5000/health
```

2.9 主要コマンド

```
# 開発
npm run dev          # 開発サーバー (ポート 5000, ホットリロード)
npm start            # 本番サーバー (ポート 8443)
npm run build        # TypeScript ビルド

# テスト
npm test             # 全テスト (10 分タイムアウト)
npm run test:unit    # ユニットテストのみ
npm run test:integration # 統合テストのみ
npm test -- tests/path/to/file.ts # 特定ファイル

# リント
npm run lint         # ESLint 実行

# MCP Services
npm run cipher-mcp   # Cipher MCP サーバー起動
claude mcp list      # MCP 状態確認
```

2.10 LLM CLI 使用方法

2.10.1 GPT-5 Codex (via Codex MCP)

```
# 基本実行
codex exec --model gpt-5-codex "Write TypeScript validator"

# 設定付き実行
codex exec --model gpt-5-codex \
  -c temperature=0.7 \
  -c max_tokens=4096 \
  --sandbox read-only \
  "Fix authentication bug"

# プロンプトベストプラクティス
# ☐ 簡潔で具体的 "Less is more"
codex exec --model gpt-5-codex "Implement OAuth2 flow"
```

```
# ☒ 冗長すぎる 不要
codex exec --model gpt-5-codex "Please write a detailed..."
```

2.10.2 Gemini 2.5 (via Gemini CLI)

```
# デフォルトモデル (gemini-2.5-flash)
gemini "Analyze system architecture"

# モデル指定
gemini --model gemini-2.5-pro "Complex analysis task"

# Deep Think モード 自動
gemini --model gemini-2.5-pro "Solve mathematical proof"
```

3 LLM Providers (Required)

HUGGINGFACE_API_KEY=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX OPENROUTER_API_KEY=sk-or-v1-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX GOOGLE_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

4 Redis (Upstash - Required)

UPSTASH_REDIS_REST_URL=https://your-redis.upstash.io UPSTASH_REDIS_REST_TOKEN=your-token-here

5 Server Configuration

PORT=4000 # Default: 4000 (dev: 5000, prod: 8443) NODE_ENV=development # development | production | test LOG_LEVEL=info # error | warn | info | debug

Optional Environment Variables

```
```bash
Anthropic API (Disabled by default)
ANTHROPIC_API_KEY=sk-ant-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
ANTHROPIC_API_ENABLED=false # Must be 'true' to enable

Gmail Integration
GMAIL_CLIENT_ID=your-client-id.apps.googleusercontent.com
GMAIL_CLIENT_SECRET=your-client-secret
GMAIL_REDIRECT_URI=https://techsapo.com/api/v1/gmail/auth/callback

Context7 Documentation Service
```

```
CONTEXT7_API_KEY=ctx7sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
Database (Optional)
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=your-password
MYSQL_DATABASE=techsapo
```

```
Cost Management
MONTHLY_BUDGET_LIMIT=70 # USD
COST_ALERT_THRESHOLD=0.8 # 80%
```

```
Logging
LOG_FILE_PATH=./logs/app.log
```

## 5.1 Production Environment

### 5.1.1 AWS Secrets Manager Setup

#### 1. Create secret in AWS

```
aws secretsmanager create-secret \
 --name techsapo/production \
 --description "TechSapo production secrets" \
 --secret-string file://secrets.json
```

#### 2. Format of secrets.json

```
{
 "HUGGINGFACE_API_KEY": "hf_xxx",
 "OPENROUTER_API_KEY": "sk-or-xxx",
 "UPSTASH_REDIS_REST_URL": "https://...",
 "UPSTASH_REDIS_REST_TOKEN": "xxx",
 "GOOGLE_API_KEY": "AIza...",
 "CONTEXT7_API_KEY": "ctx7sk-xxx"
}
```

#### 3. Configure environment

```
In production .env or systemd environment file
USE_AWS_SECRETS_MANAGER=true
AWS_SECRET_NAME=techsapo/production
AWS_REGION=us-east-1
AWS_DEFAULT_REGION=us-east-1
```

### 5.1.2 Migration Script

```
Preview migration (dry run)
npm run migrate-secrets:dry-run

Migrate secrets to AWS
npm run migrate-secrets

Update existing secrets
npm run migrate-secrets:update
```

See: [AWS Secrets Manager Setup Guide](#)

## 5.2 systemd Configuration

### 5.2.1 Important: No Quotes in .env

```
❌ WRONG - Do not use quotes in systemd .env files
UPSTASH_REDIS_REST_URL="https://your-redis.upstash.io"

\newpage

API リファレンス

Wall-Bounce エンドポイント

POST /api/v1/wall-bounce/analyze

複数 LLM による協調分析を実行。

** リクエスト **：
```json
{
  "prompt": "Analyze authentication flow",
  "domain": "analysis",
  "minProviders": 2,
  "maxRounds": 5,
  "taskType": "analysis"
}
```

レスポンス:

```
{
  "success": true,
  "result": {
    "content": " 統合された分析結果...",
    "confidence": 0.85,
```

```

    "consensus": 0.78,
    "providersUsed": ["gpt-5", "gemini-2.5-pro", "sonnet-4.5"],
    "aggregator": "sonnet-4.5",
    "rounds": 3
  },
  "metadata": {
    "totalLatencyMs": 8500,
    "estimatedCost": 0.024
  }
}

```

5.2.2 POST /api/v1/wall-bounce-sse

ストリーミング応答 Server-Sent Events .

リクエスト: 同上

レスポンス (SSE):

```

event: provider_start
data: {"provider":"gpt-5","round":1}

event: provider_result
data: {"provider":"gpt-5","content":"..."}

event: consensus_check
data: {"consensus":0.75,"threshold":0.6}

event: complete
data: {"result":{...}}

```

5.3 ヘルスチェックエンドポイント

```

# システムヘルス
curl http://localhost:8443/health

# LLM プロバイダーヘルス
curl http://localhost:8443/api/v1/llm-health

# MCP ヘルス
curl http://localhost:8443/api/v1/mcp-health

```

6 GPT-5 vs GPT-5 Codex

6.1 モデル比較

項目	GPT-5 (標準)	GPT-5 Codex
特化分野	汎用分析・推論	コーディング
Reasoning	詳細な多段階推論	Adaptive 適応推論
Thinking 詳細度		ミニマル
コード生成	可能	最適化済み
分析品質	詳細・多角的	簡潔
応答速度	中速	高速

6.2 使い分けガイド

6.2.1 GPT-5 (標準) を使うべき場合

```
# 詳細な分析
codex exec --model gpt-5 "Analyze system trade-offs"

# 文章生成
codex exec --model gpt-5 "Write API specification"

# 複雑推論
codex exec --model gpt-5 "Evaluate migration strategy"
```

6.2.2 GPT-5 Codex を使うべき場合

```
# コーディング
codex exec --model gpt-5-codex "Implement OAuth2 flow"

# デバッグ
codex exec --model gpt-5-codex "Fix: Cannot read property 'map'"

# コードレビュー
codex exec --model gpt-5-codex "Review for security issues"
```

6.3 実証結果

コーディングタスク → GPT-5 Codex 自動選択 - Model: gpt-5-codex - Thinking: ミニマル Drafting のみ - Output: 即座に高品質コード生成

分析タスク → GPT-5 (標準) 自動選択 - Model: gpt-5 - Thinking: 多段階 Structuring , Analyzing , Summarizing - Output: 詳細なセキュリティ分析

7 トラブルシューティング

7.1 一般的な問題

7.1.1 Redis 接続エラー

症状:

Error: Redis connection failed

解決:

```
# 環境変数確認
echo $UPSTASH_REDIS_REST_URL
echo $UPSTASH_REDIS_REST_TOKEN

# .env ファイル確認
cat .env | grep UPSTASH
```

7.1.2 MCP ロックファイル問題

症状:

Error: MCP server already running

解決:

```
# ロックファイル削除
rm /tmp/mcp-*.lock

# MCP 状態確認
claude mcp list
```

7.1.3 Gemini API クォータ超過

症状:

ApiError: 429 - You exceeded your current quota

解決: - Gemini 2.5 Pro にアップグレード - より高いクォータ - リクエスト間隔を調整 - バックオフ・リトライ実装済み
自動

7.1.4 GPT-5 Codex 接続失敗

症状:

Error: Codex MCP not responding

解決:

```
# Codex MCP 確認
codex exec --model gpt-5-codex "test"

# OPENAI_API_KEY 確認
echo $OPENAI_API_KEY

# Codex MCP 再起動
# ~/.config/claude/mcp.json 確認
```

7.2 ヘルスチェック手順

```
# 1. システムヘルス
curl http://localhost:8443/health
# 期待: {"status":"ok"}

# 2. LLM プロバイダーヘルス
curl http://localhost:8443/api/v1/llm-health | jq
# 期待: 全プロバイダー "available": true

# 3. MCP ヘルス
curl http://localhost:8443/api/v1/mcp-health | jq
# 期待: Codex, Context7, Cipher が "connected": true

# 4. Redis 接続
curl http://localhost:8443/api/v1/redis-health
# 期待: {"connected":true}
```

7.3 ログ確認

```
# アプリケーションログ
tail -f logs/app.log

# エラーログ
tail -f logs/app-error.log

# IT 統合ログ
tail -f logs/it-unified.log

# systemd ログ 本番
sudo journalctl -u techsapo -f
```

7.4 パフォーマンス問題

7.4.1 レスポンスが遅い

原因: - Wall-Bounce ラウンド数が多い 6 ラウンド - 複雑なプロンプト - LLM プロバイダー側のレイテンシ

解決:

```
# minProviders 削減 2→1
# maxRounds 削減 6→3
# taskType="basic" を指定 高速モード
```

7.4.2 メモリ使用量が高い

原因: - 大量の並列リクエスト - Redis キャッシュ肥大化

解決:

```
# Redis キャッシュクリア
redis-cli FLUSHDB

# Node.js メモリ制限
NODE_OPTIONS="--max-old-space-size=2048" npm start
```

8 品質保証メカニズム

8.1 コンセンサス計算

複数 LLM 応答の類似度を計算し、合意度を評価。

計算式:

$$\text{Consensus} = \Sigma(\text{similarity}(\text{response_i}, \text{response_j})) / n$$

閾値: - Consensus 0.6: 合格 - Consensus < 0.6: 追加ラウンド実行

8.2 信頼度スコア

各 LLM 応答の信頼度を評価。

評価基準: - 応答の具体性 - 構造化の度合い - 専門用語の適切な使用 - 論理的整合性

閾値: - Confidence 0.7: 高品質 - 0.5 Confidence < 0.7: 要検証 - Confidence < 0.5: 再生成

8.3 Quorum (k 値)

最小必要プロバイダー数の動的決定。

基本ルール: - コーディング: k=2 (GPT-5 Codex + Qwen3) - 分析: k=2 (GPT-5 + Gemini 2.5 Pro) - 複雑: k=3 (Gemini Deep Think + GPT-5 + Sonnet 4.5) - クリティカル: k=4 (全 Tier 動員)

8.4 Early Stopping

Quorum 達成時に即座に終了し、不要な LLM 呼び出しを回避。

条件:

IF (consensus $\geq$ 0.7 AND confidence $\geq$ 0.7 AND providers $\geq$ k):

STOP and aggregate

ELSE:

Continue to next round (max 6)

9 セキュリティ考慮事項

9.1 API キー管理

開発環境: - .env ファイルで管理 - .gitignore で除外

本番環境: - AWS Secrets Manager 推奨 - 環境変数での直接設定は非推奨

9.2 Anthropic API 制限

デフォルト: 無効

有効化方法:

```
# 環境変数で明示的に有効化
ANTHROPIC_API_ENABLED=true
ANTHROPIC_API_KEY=sk-ant-xxx
```

理由: - コスト管理 - アクセス制御 - 意図しない使用の防止

9.3 PII 保護

自動検出・マスキング: - メールアドレス - 電話番号 - クレジットカード番号 - SSN/マイナンバー

監査ログ: - 全リクエスト・レスポンスを記録 - PII 検出イベントを記録 - /audit/techdev/に保存

9.4 レート制限

プロバイダー別制限: - Gemini: 60 req/min, 1M tokens/min - OpenAI: 500 req/min, 200K tokens/min - Anthropic: 50 req/min, 100K tokens/min - OpenRouter: 200 req/min, 500K tokens/min

システム側対策: - バックオフ・リトライ - サーキットブレーカー - リクエストキューイング

10 デプロイメント

10.1 本番デプロイ手順

```
# 1. ビルド
cd /ai/prj/techdev
npm run build

# 2. 本番ディレクトリへ同期
sudo rsync -av dist/ /prod/techsapo/dist/
sudo rsync -av node_modules/ /prod/techsapo/node_modules/
sudo rsync -av package.json /prod/techsapo/

# 3. systemd サービス再起動
sudo systemctl restart techsapo

# 4. ステータス確認
sudo systemctl status techsapo

# 5. ヘルスチェック
curl http://localhost:8443/health
```

10.2 systemd 設定

場所: /etc/systemd/system/techsapo.service

設定例:

```
[Unit]
Description=TechSapo Wall-Bounce Server
After=network.target

[Service]
Type=simple
User=wombat
WorkingDirectory=/prod/techsapo
ExecStart=/data/.nvm/versions/node/v22.9.0/bin/node dist/index.js
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

10.3 監視

ログ監視:

```
# リアルタイムログ
```

```
sudo journalctl -u techsapo -f
```

```
# エラーのみ
```

```
sudo journalctl -u techsapo -p err -f
```

メトリクス: - CPU 使用率 - メモリ使用量 - レスポンスタイム - エラーレート

10.4 バックアップ

対象: - 環境変数設定 - Redis データ - 監査ログ

頻度: - 日次: Redis スナップショット - 週次: 監査ログアーカイブ

11 まとめ

11.1 システムの強み

1. 高品質: 複数 LLM の協調による高精度回答
2. 柔軟性: タスク別最適プロバイダー自動選択
3. 信頼性: コンセンサス・Quorum による品質保証
4. 拡張性: 階層化アーキテクチャで新規プロバイダー追加容易

11.2 重要原則 再掲

1. **Anthropic API** 直接使用禁止 明示的有効化時のみ
2. **GPT-5 Codex**: codex exec 経由
3. **Gemini**: gemini コマンド経由
4. 壁打ち: 最低 3 回、最大 6 回
5. コーディング: GPT-5 Codex + Qwen3 Coder 優先

11.3 次のステップ

1. 開発環境セットアップ
2. サンプルクエリ実行
3. Wall-Bounce API テスト
4. 独自タスクでの検証

11.4 リファレンス

ドキュメント: - /ai/prj/techdev/docs/ - 全ドキュメント - /ai/prj/techdev/CLAUDE.md - Claude Code
利用ガイド

リポジトリ: - /ai/prj/techdev/ - メインプロジェクト

サポート: - GitHub Issues: 問題報告・機能要望 - ドキュメント: 詳細な技術情報

作成: TechSapo Documentation Team 更新: 2025-10-07 バージョン: 2.0